Phase 1:

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

Phase 2:

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

Phase 3:

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ make pes
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes init
Initialized empty PES repository in .pes/
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ echo "hello" > file1.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ echo "world" > file2.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes add file1.txt file2.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_tree
  untracked:  tree.c
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  test_objects
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  commit.h
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  README.md
  untracked:  index.c
  untracked:  index.h
  untracked:  tree.h
```

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776665673 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776665684 6 file2.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

Phase 4:

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes init
Initialized empty PES repository in .pes/
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ echo "Hello" > hello.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes add hello.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes commit -m "Initial Commit"
Committed: 56a4067d83fc... Initial Commit
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ echo "World" >> hello.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes add hello.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes commit -m "Add World"
Committed: 4681ceafba7a... Add World
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ echo "Goodbye" > bye.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes add bye.txt
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes commit -m "Add farewell"
Committed: 002ad955ff45... Add farewell
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ ./pes log
commit 002ad955ff45ea0efeb3a8ae2c3c56a7edf8e894493ba6892fb26d8a93c43195
Author: PES User <pes@localhost>
Date:   1776666408

    Add farewell

commit 4681ceafba7a6fe41d925992f1e7527be1071df911b24a989f2a1e8df089bfc5
Author: PES User <pes@localhost>
Date:   1776666366

    Add Worldell

commit 56a4067d83fcda75fb36fa7eb17845ffaafba1d920a2986ee359b6e3f56c1e9a
Author: PES User <pes@localhost>
Date:   1776666300

    Initial Committial CommiQ

hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/00/2ad955ff45ea0efeb3a8ae2c3c56a7edf8e894493ba6892fb26d8a93c43195
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/46/81ceafba7a6fe41d925992f1e7527be1071df911b24a989f2a1e8df089bfc5
.pes/objects/56/a4067d83fcda75fb36fa7eb17845ffaafba1d920a2986ee359b6e3f56c1e9a
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ cat .pes/refs/heads/main
002ad955ff45ea0efeb3a8ae2c3c56a7edf8e894493ba6892fb26d8a93c43195
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ cat .pes/HEAD
ref: refs/heads/main
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$
```

Phase 5 and 6:

Branching and Checkout
Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

A5.1:
A branch is a file under `.pes/refs/heads/` that stores a commit hash. To implement `pes checkout <branch>`:

1. Validate that `.pes/refs/heads/<branch>` exists.
2. Read target commit hash from that branch file.
3. Resolve target commit -> root tree from object store.
4. Compare current working tree state vs index/HEAD for safety (dirty-check).
5. If safe, rewrite working directory to match target tree:
   - Create missing files/directories from target tree blobs/trees.
   - Remove tracked files/directories that do not exist in target tree.
   - Apply file modes (100644/100755).
6. Rewrite index to match checked-out tree snapshot metadata.
7. Update `.pes/HEAD` to `ref: refs/heads/<branch>`.

Why this is complex:
- Requires full tree materialization from object store into filesystem.
- Must detect and prevent data loss on local edits.
- Needs careful handling of deletes, nested directories, and file mode changes.
- Must keep HEAD, index, and working directory mutually consistent.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

A5.2:
Dirty conflict detection (using index + object store):

1. For each tracked file in index:
   - `stat()` working file.
   - If missing -> unstaged deletion.
   - If mtime/size changed, re-hash file as blob and compare with index hash.
   - If hash differs, file is dirty.
2. Resolve current HEAD tree and target branch tree.
3. For each dirty tracked file, check whether the file differs between current HEAD tree and target tree:
   - If target blob hash != current HEAD blob hash (or file added/removed in target), switching would overwrite semantic content.
4. If any such path exists, abort checkout and print conflicting paths.

This avoids false positives from timestamp-only changes and catches real overwrite risks.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

A5.3:
Detached HEAD means `.pes/HEAD` stores a commit hash directly (not `ref: ...`). New commits in this state:

- Still get created normally with parent links.
- Advance HEAD to newer detached commits.
- Are not referenced by any branch name.

Risk: user can "lose" them after moving HEAD elsewhere, because no branch keeps them reachable.

Recovery:
- If user still knows commit hash (or from reflog-like history if implemented), create a new branch file under `.pes/refs/heads/<name>` pointing to that commit.
- Then switch HEAD to that branch (`ref: refs/heads/<name>`).

Garbage Collection and Space Reclamation
Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A6.1:
Safe mark-and-sweep algorithm:

1. Build root set = all refs under `.pes/refs/heads/*` (and tags if present), plus detached HEAD hash if HEAD is direct.
2. Mark phase (graph traversal):
   - Use DFS/BFS from each root commit.
   - For each commit: mark commit object, then visit its tree and parent commit.
   - For each tree: mark tree object, then visit each entry hash (blob/tree).
   - Blobs are terminal nodes.
3. Sweep phase:
   - Enumerate all files under `.pes/objects/*/*`.
   - Delete objects not in marked set.

Efficient structure: hash set keyed by 32-byte hash (or hex string) for O(1)-average membership checks.

Scale estimate (100,000 commits, 50 branches):
- Traversal visits all reachable commits at least once, but shared history prevents multiplicative blow-up.
- Roughly O(commits + trees + blobs) reachable objects; commonly a few hundred thousand to a few million objects depending on churn and repository size.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

A6.2:
Concurrent GC vs commit race:

- Commit process may write new tree/blob/commit objects, then update branch ref at the end.
- If GC snapshots refs before branch update, new objects are not yet reachable from any ref.
- GC could sweep those just-written objects.
- Commit then updates ref to point to a commit whose subtree objects were deleted -> broken history.

How real Git avoids this (conceptually):
- Uses reachability protections and grace windows (recent objects are preserved).
- Uses lockfiles/coordination around ref updates and maintenance.
- Packs/prunes with safety constraints so objects concurrently being written are not immediately pruned.
- Reflogs and conservative prune horizons reduce accidental loss.

Test-Integration:

```
hamza@hamza-VirtualBox:~/Desktop/PES2UG24CS177-pes-vsc$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: f662d4086d9c... Initial commit

Log after first commit:
commit f662d4086d9c7567718a0c8704f140b3d90349af577f4ecf8d8f3cf770a8485e
Author: PES User <pes@localhost>
Date:   1776666706

    Initial commit


--- Second Commit ---
Committed: ba5ef55b3066... Update file.txt

--- Third Commit ---
Committed: aa336e1c98be... Add farewell

--- Full History ---
commit aa336e1c98beb7188f049d65b43e1b48099c428cd3aa983184e4b42c911e47c7
Author: PES User <pes@localhost>
Date:   1776666706

    Add farewell

commit ba5ef55b30663db8a533779c07608b3780e5e4c9b2189b40a44afd4d78df3966
Author: PES User <pes@localhost>
Date:   1776666706
```

Github Link:
https://github.com/Hamza-Sahapurwala/PES2UG24CS177-pes-vsc